```python
#!/usr/bin/env python3
"""
Aggregate individual run results into benchmark summary statistics.

Reads grading.json files from run directories and produces:
- run_summary with mean, stddev, min, max for each metric
- delta between with_skill and without_skill configurations

Usage:
   python aggregate_benchmark.py <benchmark_dir>

Example:
   python aggregate_benchmark.py benchmarks/2026-01-15T10-30-00/

The script supports two directory layouts:

   Workspace layout (from skill-creator iterations):
   <benchmark_dir>/
   └── eval-N/
       ├── with_skill/
       │   ├── run-1/grading.json
       │   └── run-2/grading.json
       └── without_skill/
           ├── run-1/grading.json
           └── run-2/grading.json

   Legacy layout (with runs/ subdirectory):
   <benchmark_dir>/
   └── runs/
       └── eval-N/
           ├── with_skill/
           │   └── run-1/grading.json
           └── without_skill/
               └── run-1/grading.json
"""

import argparse
import json
import math
import sys
from datetime import datetime, timezone
from pathlib import Path


def calculate_stats(values: list[float]) -> dict:
   """Calculate mean, stddev, min, max for a list of values."""
   if not values:
       return {"mean": 0.0, "stddev": 0.0, "min": 0.0, "max": 0.0}

   n = len(values)
   mean = sum(values) / n

   if n > 1:
       variance = sum((x - mean) ** 2 for x in values) / (n - 1)
       stddev = math.sqrt(variance)
```

```
    else:
        stddev = 0.0

    return {
        "mean": round(mean, 4),
        "stddev": round(stddev, 4),
        "min": round(min(values), 4),
        "max": round(max(values), 4)
    }


def load_run_results(benchmark_dir: Path) -> dict:
    """
    Load all run results from a benchmark directory.

    Returns dict keyed by config name (e.g. "with_skill"/"without_skill",
    or "new_skill"/"old_skill"), each containing a list of run results.
    """
    # Support both layouts: eval dirs directly under benchmark_dir, or under runs/
    runs_dir = benchmark_dir / "runs"
    if runs_dir.exists():
        search_dir = runs_dir
    elif list(benchmark_dir.glob("eval-*")):
        search_dir = benchmark_dir
    else:
        print(f"No eval directories found in {benchmark_dir} or {benchmark_dir /
'runs'}")
        return {}

    results: dict[str, list] = {}

    for eval_idx, eval_dir in enumerate(sorted(search_dir.glob("eval-*"))):
        metadata_path = eval_dir / "eval_metadata.json"
        if metadata_path.exists():
            try:
                with open(metadata_path) as mf:
                    eval_id = json.load(mf).get("eval_id", eval_idx)
            except (json.JSONDecodeError, OSError):
                eval_id = eval_idx
        else:
            try:
                eval_id = int(eval_dir.name.split("-")[1])
            except ValueError:
                eval_id = eval_idx

        # Discover config directories dynamically rather than hardcoding names
        for config_dir in sorted(eval_dir.iterdir()):
            if not config_dir.is_dir():
                continue
            # Skip non-config directories (inputs, outputs, etc.)
            if not list(config_dir.glob("run-*")):
                continue
            config = config_dir.name
            if config not in results:
                results[config] = []
```

```
        for run_dir in sorted(config_dir.glob("run-*")):
            run_number = int(run_dir.name.split("-")[1])
            grading_file = run_dir / "grading.json"

            if not grading_file.exists():
                print(f"Warning: grading.json not found in {run_dir}")
                continue

            try:
                with open(grading_file) as f:
                    grading = json.load(f)
            except json.JSONDecodeError as e:
                print(f"Warning: Invalid JSON in {grading_file}: {e}")
                continue

            # Extract metrics
            result = {
                "eval_id": eval_id,
                "run_number": run_number,
                "pass_rate": grading.get("summary", {}).get("pass_rate", 0.0),
                "passed": grading.get("summary", {}).get("passed", 0),
                "failed": grading.get("summary", {}).get("failed", 0),
                "total": grading.get("summary", {}).get("total", 0),
            }

            # Extract timing — check grading.json first, then sibling timing.json
            timing = grading.get("timing", {})
            result["time_seconds"] = timing.get("total_duration_seconds", 0.0)
            timing_file = run_dir / "timing.json"
            if result["time_seconds"] == 0.0 and timing_file.exists():
                try:
                    with open(timing_file) as tf:
                        timing_data = json.load(tf)
                    result["time_seconds"] =
timing_data.get("total_duration_seconds", 0.0)
                    result["tokens"] = timing_data.get("total_tokens", 0)
                except json.JSONDecodeError:
                    pass

            # Extract metrics if available
            metrics = grading.get("execution_metrics", {})
            result["tool_calls"] = metrics.get("total_tool_calls", 0)
            if not result.get("tokens"):
                result["tokens"] = metrics.get("output_chars", 0)
            result["errors"] = metrics.get("errors_encountered", 0)

            # Extract expectations — viewer requires fields: text, passed, evidence
            raw_expectations = grading.get("expectations", [])
            for exp in raw_expectations:
                if "text" not in exp or "passed" not in exp:
                    print(f"Warning: expectation in {grading_file} missing required
fields (text, passed, evidence): {exp}")
            result["expectations"] = raw_expectations
```

```
                # Extract notes from user_notes_summary
                notes_summary = grading.get("user_notes_summary", {})
                notes = []
                notes.extend(notes_summary.get("uncertainties", []))
                notes.extend(notes_summary.get("needs_review", []))
                notes.extend(notes_summary.get("workarounds", []))
                result["notes"] = notes

                results[config].append(result)

    return results


def aggregate_results(results: dict) -> dict:
    """
    Aggregate run results into summary statistics.

    Returns run_summary with stats for each configuration and delta.
    """
    run_summary = {}
    configs = list(results.keys())

    for config in configs:
        runs = results.get(config, [])

        if not runs:
            run_summary[config] = {
                "pass_rate": {"mean": 0.0, "stddev": 0.0, "min": 0.0, "max": 0.0},
                "time_seconds": {"mean": 0.0, "stddev": 0.0, "min": 0.0, "max": 0.0},
                "tokens": {"mean": 0, "stddev": 0, "min": 0, "max": 0}
            }
            continue

        pass_rates = [r["pass_rate"] for r in runs]
        times = [r["time_seconds"] for r in runs]
        tokens = [r.get("tokens", 0) for r in runs]

        run_summary[config] = {
            "pass_rate": calculate_stats(pass_rates),
            "time_seconds": calculate_stats(times),
            "tokens": calculate_stats(tokens)
        }

    # Calculate delta between the first two configs (if two exist)
    if len(configs) >= 2:
        primary = run_summary.get(configs[0], {})
        baseline = run_summary.get(configs[1], {})
    else:
        primary = run_summary.get(configs[0], {}) if configs else {}
        baseline = {}

    delta_pass_rate = primary.get("pass_rate", {}).get("mean", 0) -
baseline.get("pass_rate", {}).get("mean", 0)
    delta_time = primary.get("time_seconds", {}).get("mean", 0) -
baseline.get("time_seconds", {}).get("mean", 0)
```

```
    delta_tokens = primary.get("tokens", {}).get("mean", 0) - baseline.get("tokens",
{}).get("mean", 0)

    run_summary["delta"] = {
        "pass_rate": f"{delta_pass_rate:+.2f}",
        "time_seconds": f"{delta_time:+.1f}",
        "tokens": f"{delta_tokens:+.0f}"
    }

    return run_summary


def generate_benchmark(benchmark_dir: Path, skill_name: str = "", skill_path: str =
"") -> dict:
    """
    Generate complete benchmark.json from run results.
    """
    results = load_run_results(benchmark_dir)
    run_summary = aggregate_results(results)

    # Build runs array for benchmark.json
    runs = []
    for config in results:
        for result in results[config]:
            runs.append({
                "eval_id": result["eval_id"],
                "configuration": config,
                "run_number": result["run_number"],
                "result": {
                    "pass_rate": result["pass_rate"],
                    "passed": result["passed"],
                    "failed": result["failed"],
                    "total": result["total"],
                    "time_seconds": result["time_seconds"],
                    "tokens": result.get("tokens", 0),
                    "tool_calls": result.get("tool_calls", 0),
                    "errors": result.get("errors", 0)
                },
                "expectations": result["expectations"],
                "notes": result["notes"]
            })

    # Determine eval IDs from results
    eval_ids = sorted(set(
        r["eval_id"]
        for config in results.values()
        for r in config
    ))

    benchmark = {
        "metadata": {
            "skill_name": skill_name or "<skill-name>",
            "skill_path": skill_path or "<path/to/skill>",
            "executor_model": "<model-name>",
            "analyzer_model": "<model-name>",
```

```
            "timestamp": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
            "evals_run": eval_ids,
            "runs_per_configuration": 3
        },
        "runs": runs,
        "run_summary": run_summary,
        "notes": []  # To be filled by analyzer
    }

    return benchmark


def generate_markdown(benchmark: dict) -> str:
    """Generate human-readable benchmark.md from benchmark data."""
    metadata = benchmark["metadata"]
    run_summary = benchmark["run_summary"]

    # Determine config names (excluding "delta")
    configs = [k for k in run_summary if k != "delta"]
    config_a = configs[0] if len(configs) >= 1 else "config_a"
    config_b = configs[1] if len(configs) >= 2 else "config_b"
    label_a = config_a.replace("_", " ").title()
    label_b = config_b.replace("_", " ").title()

    lines = [
        f"# Skill Benchmark: {metadata['skill_name']}",
        "",
        f"**Model**: {metadata['executor_model']}",
        f"**Date**: {metadata['timestamp']}",
        f"**Evals**: {', '.join(map(str, metadata['evals_run']))}
({metadata['runs_per_configuration']} runs each per configuration)",
        "",
        "## Summary",
        "",
        f"| Metric | {label_a} | {label_b} | Delta |",
        "|--------|------------|---------------|-------|",
    ]

    a_summary = run_summary.get(config_a, {})
    b_summary = run_summary.get(config_b, {})
    delta = run_summary.get("delta", {})

    # Format pass rate
    a_pr = a_summary.get("pass_rate", {})
    b_pr = b_summary.get("pass_rate", {})
    lines.append(f"| Pass Rate | {a_pr.get('mean', 0)*100:.0f}% ± {a_pr.get('stddev',
0)*100:.0f}% | {b_pr.get('mean', 0)*100:.0f}% ± {b_pr.get('stddev', 0)*100:.0f}% |
{delta.get('pass_rate', '—')} |")

    # Format time
    a_time = a_summary.get("time_seconds", {})
    b_time = b_summary.get("time_seconds", {})
    lines.append(f"| Time | {a_time.get('mean', 0):.1f}s ± {a_time.get('stddev',
0):.1f}s | {b_time.get('mean', 0):.1f}s ± {b_time.get('stddev', 0):.1f}s |
{delta.get('time_seconds', '—')}s |")
```

```
    # Format tokens
    a_tokens = a_summary.get("tokens", {})
    b_tokens = b_summary.get("tokens", {})
    lines.append(f"| Tokens | {a_tokens.get('mean', 0):.0f} ± {a_tokens.get('stddev',
0):.0f} | {b_tokens.get('mean', 0):.0f} ± {b_tokens.get('stddev', 0):.0f} |
{delta.get('tokens', '—')} |")

    # Notes section
    if benchmark.get("notes"):
        lines.extend([
            "",
            "## Notes",
            ""
        ])
        for note in benchmark["notes"]:
            lines.append(f"- {note}")

    return "\n".join(lines)


def main():
    parser = argparse.ArgumentParser(
        description="Aggregate benchmark run results into summary statistics"
    )
    parser.add_argument(
        "benchmark_dir",
        type=Path,
        help="Path to the benchmark directory"
    )
    parser.add_argument(
        "--skill-name",
        default="",
        help="Name of the skill being benchmarked"
    )
    parser.add_argument(
        "--skill-path",
        default="",
        help="Path to the skill being benchmarked"
    )
    parser.add_argument(
        "--output", "-o",
        type=Path,
        help="Output path for benchmark.json (default: <benchmark_dir>/benchmark.json)"
    )

    args = parser.parse_args()

    if not args.benchmark_dir.exists():
        print(f"Directory not found: {args.benchmark_dir}")
        sys.exit(1)

    # Generate benchmark
    benchmark = generate_benchmark(args.benchmark_dir, args.skill_name,
args.skill_path)
```

```
    # Determine output paths
    output_json = args.output or (args.benchmark_dir / "benchmark.json")
    output_md = output_json.with_suffix(".md")

    # Write benchmark.json
    with open(output_json, "w") as f:
        json.dump(benchmark, f, indent=2)
    print(f"Generated: {output_json}")

    # Write benchmark.md
    markdown = generate_markdown(benchmark)
    with open(output_md, "w") as f:
        f.write(markdown)
    print(f"Generated: {output_md}")

    # Print summary
    run_summary = benchmark["run_summary"]
    configs = [k for k in run_summary if k != "delta"]
    delta = run_summary.get("delta", {})

    print(f"\nSummary:")
    for config in configs:
        pr = run_summary[config]["pass_rate"]["mean"]
        label = config.replace("_", " ").title()
        print(f"  {label}: {pr*100:.1f}% pass rate")
    print(f"  Delta:         {delta.get('pass_rate', '—')}")


if __name__ == "__main__":
    main()
```